



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding with C#

TGS Ref No: TGS-2023039178

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version v7.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
6.0	Date not stated in original PPT	Legacy content reference; original instructional topics retained.	Original source
7.0	13 September 2026	Rebuilt visual deck; corrected stale claims and wrong-course LOs; added ten C# activities, vibe prompts, tests, design and documentation depth. PPT/LG/LP/activities revised.	Dr Alfred Ang

TABLE OF CONTENTS

Course information.....	4
Learning outcomes and competency criteria.....	4
Environment setup and review workflow.....	4
Topic 1 — AI-assisted C# component design.....	5
Topic 2 — C# programming fundamentals.....	11
Topic 3 — Classes, objects and design controls.....	24
Topic 4 — Functionality and interoperability.....	32
Topic 5 — Design documentation and handover.....	38
Assessment preparation and submission.....	44
Source register and C++ comparison boundary.....	44
Glossary.....	45

Course information

AI Vibe Coding with C# · TGS-2023039178 · Software Design Level 3 · ICT-DES-3005-1.1 · v7.0 · 13 September 2026. Approved programme: two days / 16 contact hours, comprising 7 classroom hours, 7 practical hours, 1 hour WA and 1 hour PP. Lunch is excluded.

The original PPT is the content authority; the current Tertiary all-white component system is the design authority. The five learning outcomes below replace an unrelated marketing page in the original. AI is optional; every project runs locally without credentials.

Registration: <https://www.tertiarycourses.com.sg/wsqa-vibe-coding-with-c-sharp.html>

Learning outcomes and competency criteria

Outcome	Knowledge	Ability
LO1: Determine basic software components using C# methodologies to meet functional specifications.	K1: Design requirements for simple, basic software components	A1: Design a simple software component or interface according to functional specifications and business requirements
LO2: Apply C# methodologies and tools for software creation.	K2: Basic software design tools and techniques	A2: Utilise appropriate software design methods and tools, in line with the organisation's software design practice and principles
LO3: Select essential C# controls and features to meet software design requirements.	K3: Types of controls, elements and features in software	A3: Identify relevant controls, elements and features to be included in the software to meet its design objectives
LO4: Examine the interoperability and functionality of C# software components.	K4: Indicators of software functionality and interoperability	A4: Assess functionality and interoperability of different elements or components in the software design
LO5: Generate C# design documentation aligned with user specifications.	K5: Documentation of design details	A5: Produce detailed design documentation mapped to user specifications

Environment setup and review workflow

1. Install the .NET 10 SDK from Microsoft for your operating system. Install VS Code; C# Dev Kit is optional for IntelliSense/debugging.

2. Download or clone the learner repository. Open one activity folder at a time; each contains Activity.csproj and Program.cs.

```
dotnet --list-sdks
dotnet build Activity.csproj
dotnet run --project Activity.csproj
dotnet run --project Activity.csproj -- --self-test
```

3. Read requirements.md, expected-output.txt and prompts.pdf. Run the supplied baseline, then make one reviewed change at a time.

4. Record the actual compiler output, test inputs, expected/actual results and the inspected diff. AI suggestions must be explained and tested; never use synthetic AI test claims as evidence.

5. Use the same terminal commands on Windows, macOS or Linux. A runtime alone is insufficient. NETSDK1045 means the selected SDK is too old for the net10.0 target.

6. To debug in VS Code, open Program.cs, place a breakpoint at the business rule, start a C# debug session and inspect locals/call stack. Compare normal and invalid inputs. Use terminal execution as the acceptance evidence.

7. For an AI review, provide only the synthetic specification and minimal code/error excerpt. Ask for a design, smallest diff and boundary tests. Read the diff, reject unrelated files/dependencies, apply the accepted edit and rerun checks. A manual review follows the same evidence path.

Topic 1 — AI-assisted C# component design

Deck slides 15–50; maps to K1, A1 and LO1.

Source, assembly and runtime boundaries

Slides 17–18. C# compiles into a managed assembly; the runtime loads its referenced libraries.

```
// Program.cs -> compiler -> assembly -> CLR
Console.WriteLine(Environment.Version);
Console.WriteLine(typeof(string).Assembly.GetName().Name);
```

Failure to diagnose: An installed runtime without an SDK cannot build source.

Design decision: Use modern .NET for cross-platform examples; .NET Framework is a distinct legacy Windows product.

Original source coverage: slides 20, 21, 22, 23. These subjects have been recomposed and corrected in the mechanism above.

A project is a reproducible compiler contract

Slides 19–20. The project fixes the API target and makes nullable warnings visible.

```
<TargetFramework>net10.0</TargetFramework>
<Nullable>enable</Nullable>
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
```

Failure to diagnose: An older SDK cannot compile a newer target framework.

Design decision: Keep project settings under version control; never accept an AI retargeting change without reviewing its impact.

Original source coverage: slides 24, 27. These subjects have been recomposed and corrected in the mechanism above.

One console entry point

Slides 21–22. The executable starts once and prints a deterministic message.

```
using System;
Console.WriteLine("Hello, warehouse!");
// Top-level statements generate the entry point.
```

Failure to diagnose: Combining top-level statements with another entry point causes a compiler warning/error.

Design decision: Prefer a small portable console entry point; keep business rules in separate methods.

Original source coverage: slides 25, 26, 28. These subjects have been recomposed and corrected in the mechanism above.

Identifiers and keyword escaping

Slides 23–24. Underscore may start an identifier; @ escapes a keyword identifier.

```
int _stock = 10;
string productName = "Rice";
string @class = "Food";
Console.WriteLine($"{productName}: {_stock}");
```

Failure to diagnose: Names starting with a digit or containing a space are invalid.

Design decision: Use PascalCase for public members and camelCase for locals; prefer clear names to escaped keywords.

Original source coverage: slides 29, 30. These subjects have been recomposed and corrected in the mechanism above.

A functional requirement becomes an invariant

Slides 25–26. A precise acceptance example exposes the forbidden price boundary.

```
string? error = Validate("Rice", 0m, 1, "Food");
// R2: price must be strictly greater than 0.
// Expected: "price"; no record is created.
```

Failure to diagnose: A vague requirement such as good validation has no testable pass condition.

Design decision: Give each requirement an ID, concrete inputs and a measurable expected result.

Separate business rules from user interface controls

Slides 27–28. The same invariant survives replacing a console view with a GUI.

```
// View collects text -> parser -> Product constructor.
// Product enforces Price > 0 and Stock >= 0.
// View renders success only after Add succeeds.
```

Failure to diagnose: UI-only validation can be bypassed by another component calling the model.

Design decision: Keep validation at the component boundary and provide user-friendly feedback in the View.

An AI prompt is a bounded design contract

Slides 29–30. Constraints let the reviewer compare the proposed code against the same specification.

```
// Specification: R1 name required; R2 price > 0.
// Target: C#, .NET 10, no new dependencies.
// Output: design, minimal diff, boundary tests.
// Evidence: actual compiler/run output only.
```

Failure to diagnose: An unbounded prompt may introduce packages, destructive operations or invented APIs.

Design decision: Approve a concrete diff only after explaining and testing it.

Review an AI patch against a failing example

Slides 31–32. One boundary input proves why the proposed operator must change.

```
bool invalid = price < 0; // candidate bug
bool repaired = price <= 0; // correct R2 guard
// price=0: invalid False, repaired True.
```

Failure to diagnose: An AI assertion that code is correct is not execution evidence.

Design decision: Record input, expected/actual output, inspected diff and rerun result.

Compile-time, runtime and business failures differ

Slides 33–34. Different evidence points to a different repair layer.

```
// CS0103: misspelled identifier -> compile failure.  
// FormatException: invalid text -> runtime failure.  
// price=0 accepted -> business rule failure.
```

Failure to diagnose: Fixing a compiler error does not prove the business invariant.

Design decision: Require successful build plus normal, invalid and boundary tests.

C# and C++ share syntax, not runtime contracts

Slides 35–36. The activities compile as C# projects and use the .NET library.

```
// C#: List<int> quantities = new() {10,20};  
// C++ vector<int> is a comparison concept only.  
// C# uses using/IDisposable for owned resources.
```

Failure to diagnose: Pasting #include, std::cout or native pointer ownership into C# fails its language/runtime contract.

Design decision: Use the C++ links only to compare ideas; verify every C# API with Microsoft documentation.

UML names a concrete component boundary

Slides 37–38. A class diagram shows ownership, visibility and relationships before code is expanded.

```
// ProductCatalogue owns List<Product>.  
// Product validates its constructor inputs.  
// CatalogueView formats confirmation.  
// Add() mutates the catalogue after validation.
```

Failure to diagnose: A diagram that omits mutation ownership leaves invariant enforcement unclear.

Design decision: Label properties, public methods and multiplicity; map the boundary to requirement IDs.

Pseudocode and flowchart expose rejection paths

Slides 39–40. Ordering the guards makes the first rejected rule and success path clear.

```
// IF name blank: reject R1  
// ELSE IF price <= 0: reject R2  
// ELSE IF stock < 0: reject R3  
// ELSE add and display confirmation
```

Failure to diagnose: Drawing only the happy path misses all invalid-input behaviour.

Design decision: Represent decisions and outcomes, then test at least one input for each branch.

Activity 01 — From catalogue specification to executable contract

LO1 · K1 · A1; deck slides 41–45. Folder: activities/activity-01-specification-contract.

A store needs a catalogue for Electronics, Clothing and Food. Reject blank names, prices at or below zero, negative stock and unknown categories. Define each requirement before generating a component.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A store needs a catalogue for Electronics, Clothing and Food. Reject blank names, prices at or below zero, negative stock and unknown categories. Define each requirement before generating a component. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Write a requirements table with R1–R3, one example and one boundary case each.
- Add a maximum name length rule and show its boundary tests.
- Draw the validator/catalogue/view component boundary and identify who may mutate stock.

Acceptance checks and expected output

- R1: blank names fail without creating a record.
- R2: price must be strictly positive; stock zero is valid.
- R3: only three named categories are accepted.
- Self-test reports PASS: 5 contract checks.

```
Valid catalogue record: True
Invalid price -> price
```

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Activity 02 — Compile, run and review an AI-generated C# component

LO1 · K1 · A1; deck slides 46–50. Folder: activities/activity-02-ai-review-hello.

A greeter formats a warehouse welcome message. AI-generated text is a candidate change; compile it, test null/blank input and explain the result before accepting it.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A greeter formats a warehouse welcome message. AI-generated text is a candidate change; compile it, test null/blank input and explain the result before accepting it. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Ask AI for an explanation of top-level statements, using only this file.
- Deliberately remove the null/blank guard, observe a failure, then restore it.
- Add a name containing Unicode and show that it is preserved.

Acceptance checks and expected output

- Whitespace and null input produce the learner fallback.

- A non-empty name is trimmed before formatting.
- Program has one entry point and compiles without warnings.
- Self-test reports PASS: 3 greeting checks.

Welcome, Ada!
Welcome, learner!

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Topic 2 — C# programming fundamentals

Deck slides 51–137; maps to K2, A2 and LO2.

Value types copy the value

Slides 52–53. Changing the copy leaves the original integer at 10.

```
int stock = 10;
int copy = stock;
copy = 20;
Console.WriteLine(stock); // 10
```

Failure to diagnose: Assuming two value variables share one storage location gives incorrect updates.

Design decision: Use int for bounded counts, decimal for currency and bool for conditions.

Original source coverage: slides 32, 33, 34. These subjects have been recomposed and corrected in the mechanism above.

References can share a mutable object

Slides 54–55. Both variables reference the same list; mutation is visible through either.

```
var a = new List<int> { 10 };
var b = a;
b.Add(20);
Console.WriteLine(a.Count); // 2
```

Failure to diagnose: Returning a mutable internal list lets callers bypass validation.

Design decision: Expose a read-only view or deliberate copy at a component boundary.

Original source coverage: slides 35. These subjects have been recomposed and corrected in the mechanism above.

Boxing and unboxing preserve runtime type

Slides 56–57. A boxed int must be unboxed as int before a numeric conversion.

```
object boxed = 10;
int quantity = (int)boxed;
// (long)boxed would fail: boxed value is an int.
Failure to diagnose: Unboxing to the wrong value type throws InvalidCastException.
```

Design decision: Prefer typed generic collections to object-based containers.

Original source coverage: slides 36. These subjects have been recomposed and corrected in the mechanism above.

var and dynamic resolve at different times

Slides 58–59. var retains static checking; dynamic defers binding until execution.

```
var quantity = 10; // compile-time int
dynamic candidate = 10;
Console.WriteLine(candidate + 2); // 12
Failure to diagnose: A nonexistent dynamic member fails at runtime instead of compile time.
```

Design decision: Keep domain components statically typed; confine dynamic to explicit interoperability boundaries.

Original source coverage: slides 37. These subjects have been recomposed and corrected in the mechanism above.

Strings are immutable reference values

Slides 60–61. Formatting returns a new string and leaves the original unchanged.

```
string original = "Rice";
string upper = original.ToUpperInvariant();
Console.WriteLine(original); // Rice
Console.WriteLine(upper); // RICE
Failure to diagnose: Treating string mutation as in-place mutation loses the returned result.
```

Design decision: Use invariant casing for identifiers and explicit culture for user-facing text.

Original source coverage: slides 38, 53. These subjects have been recomposed and corrected in the mechanism above.

Unsafe pointers are an advanced boundary

Slides 62–63. Pointer dereferencing reads the pointed-to integer.

```
// Concept excerpt; requires AllowUnsafeBlocks=true.
unsafe {
    int value = 10; int* address = &value;
    Console.WriteLine(*address);
}
```

Failure to diagnose: Unsafe code can violate memory safety and requires explicit compiler permission.

Design decision: The learner activities use safe managed types; pointers are retained as legacy coverage, not a required assessment tool.

Original source coverage: slides 39. These subjects have been recomposed and corrected in the mechanism above.

Implicit numeric conversion can lose precision

Slides 64–65. Widening an integer range differs from preserving every significant digit.

```
int quantity = 10;
long wide = quantity;
int large = 16_777_217;
float approximate = large;
Console.WriteLine((int)approximate == large); // False
Failure to diagnose: int to float can round large exact integers.
```

Design decision: Choose types using the value range and precision contract, not only storage size.

Original source coverage: slides 40, 41. These subjects have been recomposed and corrected in the mechanism above.

Casts and conversion methods have distinct contracts

Slides 66–67. An explicit cast truncates this positive value; Convert rounds it.

```
double value = 2.9;
int truncated = (int)value; // 2
int rounded = Convert.ToInt32(value); // 3
Console.WriteLine($"{truncated}, {rounded}");
Failure to diagnose: Using the wrong conversion silently changes a business calculation.
```

Design decision: Document the rounding rule and test midpoint and negative values.

Original source coverage: slides 42, 43. These subjects have been recomposed and corrected in the mechanism above.

Definite assignment protects reads

Slides 68–69. A local must be assigned on every path before its value is read.

```
int quantity;
quantity = 10;
Console.WriteLine(quantity);
// Reading before assignment is a compiler error.
Failure to diagnose: A conditionally assigned variable may leave an unassigned branch.
```

Design decision: Initialize deliberately; avoid default values that hide missing required input.

Original source coverage: slides 44, 45, 46. These subjects have been recomposed and corrected in the mechanism above.

External text enters through a parsing guard

Slides 70–71. TryParse reports malformed input without throwing for normal bad text.

```
string input = "two";
bool parsed = int.TryParse(input, out int quantity);
Console.WriteLine(parsed); // False
// Do not use quantity unless parsed is true.
Failure to diagnose: Parse on untrusted input can interrupt the workflow with FormatException.
```

Design decision: Validate syntax, range and domain invariant as separate controls.

Original source coverage: slides 47. These subjects have been recomposed and corrected in the mechanism above.

Assignment target and evaluated expression

Slides 72–73. The right side is evaluated before it is stored in the target.

```
int quantity = 10;
quantity = quantity + 2;
// quantity is assignable; quantity+2 is a value.
Console.WriteLine(quantity); // 12
```

Failure to diagnose: A calculation expression cannot be assigned to as if it were a variable.

Design decision: Use property setters/methods where assignments must preserve invariants.

Original source coverage: slides 48. These subjects have been recomposed and corrected in the mechanism above.

Literal suffixes define numeric intent

Slides 74–75. The m suffix makes the price decimal rather than double.

```
int stock = 10;
long large = 10_000_000_000L;
decimal price = 19.90m;
double ratio = 0.1;
```

Failure to diagnose: Mixing decimal and double requires an explicit conversion.

Design decision: Keep money calculations in decimal and decide rounding at a defined boundary.

Original source coverage: slides 49, 50. These subjects have been recomposed and corrected in the mechanism above.

Characters, escaping and Unicode

Slides 76–77. Escapes represent control characters; Unicode text survives managed string handling.

```
char newline = '\n';
string path = @"C:\data\stock.json";
string label = "米";
Console.WriteLine(label);
```

Failure to diagnose: A char is one UTF-16 code unit and need not be a complete visible character.

Design decision: Use strings for names; preserve UTF-8 encoding in exported text files.

Original source coverage: slides 51, 52. These subjects have been recomposed and corrected in the mechanism above.

Constants and readonly values

Slides 78–79. const expresses a compile-time fixed value.

```
const decimal TaxRate = 0.09m;
decimal price = 10m;
decimal tax = price * TaxRate;
Console.WriteLine(tax); // 0.90
```

Failure to diagnose: A const cannot depend on a value loaded at runtime.

Design decision: Use readonly for constructor-initialized state; keep time-sensitive business rates configurable.

Original source coverage: slides 54. These subjects have been recomposed and corrected in the mechanism above.

Arithmetic and integer division

Slides 80–81. Operand types determine division behaviour; remainder has a separate contract.

```
Console.WriteLine(5 / 2); // 2
Console.WriteLine(5m / 2); // 2.5
Console.WriteLine(5 % 2); // 1
```

Failure to diagnose: Dividing two ints before converting to decimal loses the fractional part.

Design decision: Convert before dividing and guard zero denominators.

Original source coverage: slides 55, 56. These subjects have been recomposed and corrected in the mechanism above.

Relational operators encode boundaries

Slides 82–83. Zero stock is valid; zero price is invalid in the catalogue specification.

```
int quantity = 0;
Console.WriteLine(quantity >= 0); // True
decimal price = 0m;
Console.WriteLine(price > 0); // False
```

Failure to diagnose: Using >= instead of > for price admits a forbidden boundary.

Design decision: Translate words such as strictly positive into exact comparison operators.

Original source coverage: slides 57. These subjects have been recomposed and corrected in the mechanism above.

Short-circuit logic prevents invalid reads

Slides 84–85. && skips the second operand when the first is false.

```
string? name = null;
bool valid = name is not null && name.Length > 0;
Console.WriteLine(valid); // False
```

Failure to diagnose: Using & evaluates both operands and may dereference null.

Design decision: Put the safety guard before the dependent access.

Original source coverage: slides 58. These subjects have been recomposed and corrected in the mechanism above.

Bitwise flags are different from booleans

Slides 86–87. Flag values are independent bits combined with OR.

```
[Flags] enum Permission { None=0, Read=1, Write=2 }
// Within a method:
// var p = Permission.Read | Permission.Write;
// (p & Permission.Write) != 0 is true.
```

Failure to diagnose: Sequential enum values are not a valid independent-bit design.

Design decision: Keep permission modelling separate from actual server-side authorization enforcement.

Original source coverage: slides 59. These subjects have been recomposed and corrected in the mechanism above.

Assignment, increment and checked arithmetic

Slides 88–89. Post-increment returns the old value; checked catches overflow.

```
int quantity = 10;
quantity += 2;
int before = quantity++; // 12
Console.WriteLine(quantity); // 13
int safe = checked(quantity + 1);
```

Failure to diagnose: An unchecked count overflow can produce a negative value.

Design decision: Avoid complex increment expressions; validate before committing a stock mutation.

Original source coverage: slides 60. These subjects have been recomposed and corrected in the mechanism above.

Null and conditional operators

Slides 90–91. ?? chooses a null fallback; ?: chooses one expression from a boolean condition.

```
string? supplied = null;
string name = supplied ?? "learner";
int stock = 0;
string state = stock > 0 ? "available" : "empty";
```

Failure to diagnose: A fallback for null does not treat blank whitespace as missing.

Design decision: Use IsNullOrWhiteSpace when the domain contract rejects empty names.

Original source coverage: slides 61, 64, 81. These subjects have been recomposed and corrected in the mechanism above.

Operator precedence changes the calculation

Slides 92–93. Multiplication binds more tightly than addition.

```
decimal a = 10m + 2m * 3m; // 16
decimal b = (10m + 2m) * 3m; // 36
Console.WriteLine($"{a}, {b}");
```

Failure to diagnose: A missing parenthesis changes a discount or tax base.

Design decision: Use explicit parentheses for business formulas and test a worked numeric example.

Original source coverage: slides 62. These subjects have been recomposed and corrected in the mechanism above.

if, else and switch choose one path

Slides 94–95. The first matching grade band handles 80 as B.

```
int grade = 80;
string band = grade switch {
    >=90 => "A", >=80 => "B",
```

```
>=70 => "C", >=60 => "D", _ => "F"
};
```

Failure to diagnose: Placing a broad ≥ 60 case first hides higher bands.

Design decision: Order overlapping rules from specific/high thresholds to broader cases.

Original source coverage: slides 63. These subjects have been recomposed and corrected in the mechanism above.

Loop termination is part of correctness

Slides 96–97. The changing counter reaches the stopping condition.

```
int remaining = 3;
while (remaining > 0) {
    Console.WriteLine(remaining);
    remaining--;
}
```

Failure to diagnose: A loop that never changes its condition becomes infinite.

Design decision: Bound retries and make the termination measure explicit.

Original source coverage: slides 65, 67. These subjects have been recomposed and corrected in the mechanism above.

break and continue have different effects

Slides 98–99. continue skips one iteration; break ends the nearest loop.

```
for (int i=0; i<5; i++) {
    if (i==1) continue;
    if (i==3) break;
    Console.WriteLine(i);
} // prints 0, 2
```

Failure to diagnose: Using continue to terminate a loop leaves later iterations running.

Design decision: Keep the skip and stop business conditions distinct.

Original source coverage: slides 66. These subjects have been recomposed and corrected in the mechanism above.

Method inputs, outputs and responsibility

Slides 100–101. A pure method calculates a result from explicit inputs.

```
static decimal Discount(decimal price, decimal rate) {
    if (rate<0 || rate>1) throw new ArgumentOutOfRangeException(nameof(rate));
    return price * (1-rate);
}
```

Failure to diagnose: Hidden global state makes tests depend on execution order.

Design decision: Name a method for one responsibility and specify valid inputs and return meaning.

Original source coverage: slides 74, 75, 76, 77. These subjects have been recomposed and corrected in the mechanism above.

Recursion needs a base case and input bound

Slides 102–103. The base case returns 1; Factorial(5) returns 120.

```
static int Factorial(int n) {
    if(n<0 || n>12) throw new ArgumentOutOfRangeException(nameof(n));
    return n<=1 ? 1 : checked(n*Factorial(n-1));
}
```

Failure to diagnose: Missing the base case causes unbounded recursion; large results overflow int.

Design decision: Prefer iteration when a simple counter expresses the same rule with less stack use.

Original source coverage: slides 78. These subjects have been recomposed and corrected in the mechanism above.

Value, ref and out parameters

Slides 104–105. ref passes a writable storage location; out produces a value on each successful return path.

```
static void Increment(ref int count) => count++;
int quantity = 10;
Increment(ref quantity); // quantity is 11
bool ok = int.TryParse("12", out int parsed);
```

Failure to diagnose: Assuming a normal int parameter updates the caller leaves its value unchanged.

Design decision: Prefer return values unless a ref/out contract has a clear purpose.

Original source coverage: slides 79. These subjects have been recomposed and corrected in the mechanism above.

Nullable values represent absence

Slides 106–107. Nullable value types distinguish missing stock from zero stock.

```
int? stock = null;
Console.WriteLine(stock.HasValue); // False
int displayStock = stock ?? 0;
// null means unknown; 0 means known empty.
```

Failure to diagnose: Calling Value when HasValue is false throws.

Design decision: Do not collapse unknown into zero unless the display/use contract permits it.

Original source coverage: slides 80. These subjects have been recomposed and corrected in the mechanism above.

Array rank and length are different

Slides 108–109. Rank counts dimensions; Length counts all elements.

```
int[,] grid = new int[2,3];
Console.WriteLine(grid.Rank); // 2
Console.WriteLine(grid.Length); // 6
Console.WriteLine(grid.GetLength(0)); // 2
```

Failure to diagnose: Confusing rank with size breaks indexing and traversal.

Design decision: Use zero-based indexes and compare each dimension against its own length.

Original source coverage: slides 82, 83, 84, 85. These subjects have been recomposed and corrected in the mechanism above.

foreach and array variants

Slides 110–111. foreach visits each value; casting before division gives mean 80.

```
int[] grades = [90,80,70];  
int sum = 0;  
foreach(int grade in grades) sum += grade;  
decimal mean = (decimal)sum / grades.Length;  
Failure to diagnose: An empty array causes division by zero.
```

Design decision: Use an array for fixed shape and List<T> for changing counts.

Original source coverage: slides 86, 87. These subjects have been recomposed and corrected in the mechanism above.

Constructing and formatting strings

Slides 112–113. Interpolation combines values with explicit numeric formatting.

```
string product = "Rice";  
decimal price = 2.50m;  
string line = $"{product}: {price:F2}";  
Console.WriteLine(line);  
Failure to diagnose: Culture-dependent output can break machine-to-machine comparisons.
```

Design decision: Use explicit/invariant culture for exchange formats and the user's locale for display.

Original source coverage: slides 88, 89, 90. These subjects have been recomposed and corrected in the mechanism above.

Ordinal comparison and search

Slides 114–115. The comparison contract is explicitly case-insensitive and ordinal.

```
string title = "C# Design";  
bool match = title.Contains("c#", StringComparison.OrdinalIgnoreCase);  
Console.WriteLine(match); // True  
Failure to diagnose: Default culture-sensitive comparisons may surprise identifier matching.
```

Design decision: Select comparison mode according to human-language versus identifier semantics.

Original source coverage: slides 91, 92. These subjects have been recomposed and corrected in the mechanism above.

Substring, join and range guards

Slides 116–117. Substring selects a bounded range; Join inserts a delimiter between values.

```
string sku = "SKU-001";  
string suffix = sku.Substring(4,3);  
string joined = string.Join(", ", new[] { "Rice", "Tea" });  
Failure to diagnose: Substring throws when start/length exceed bounds.
```

Design decision: Validate the format before slicing external text.

Original source coverage: slides 93, 94. These subjects have been recomposed and corrected in the mechanism above.

MVC makes coordination explicit

Slides 118–119. The Controller connects a domain value to its display without duplicating either responsibility.

```
// Model: BankAccount.Balance
// View: BalanceView.Format(number,balance)
// Controller: BalanceController.Enquire()
// Result: DEMO-001: balance 125.50
```

Failure to diagnose: Putting balance validation in the View creates inconsistent business rules.

Design decision: Use the console MVC example as a small design technique before adopting a web framework.

Boundary tests target the exact grade thresholds

Slides 120–121. Adjacent inputs reveal off-by-one comparison errors.

```
// Band(89) -> B; Band(90) -> A
// Band(79) -> C; Band(80) -> B
// Band(59) -> F; Band(60) -> D
```

Failure to diagnose: Testing only 95 never distinguishes >90 from >=90.

Design decision: For each threshold T, test T-1, T and T+1 where the input domain permits.

Money rounding belongs to a named boundary

Slides 122–123. A documented rounding rule makes receipt output reproducible.

```
decimal total = 19.90m * 2 * (1-0.10m);
decimal receipt = decimal.Round(total,2,MidpointRounding.AwayFromZero);
// Receipt: 35.82
```

Failure to diagnose: Rounding every intermediate step can differ from rounding the final total.

Design decision: State where rounding occurs and test a midpoint such as 1.005m.

A nullable reference warning reveals missing handling

Slides 124–125. The guard establishes a usable non-empty label without disabling nullable analysis.

```
string? input = null;
string label = string.IsNullOrEmpty(input)
    ? "learner" : input.Trim();
```

Failure to diagnose: Adding ! suppresses the warning but does not make a null value safe.

Design decision: Treat nullability annotations as an input contract; repair the path rather than suppressing it.

Collection shape determines the update contract

Slides 126–127. An array has fixed length; a List grows through an explicit operation.

```
int[] fixedGrades = [90,80,70];
var growingProducts = new List<string>();
growingProducts.Add("Rice");
```

Failure to diagnose: Choosing an array then assuming Add exists breaks the API expectation.

Design decision: Match the container to fixed dimensions, growing collections and lookup requirements.

Activity 03 — Types, operators, input rules and batch control flow

LO2 · K2 · A2; deck slides 128–132. Folder: activities/activity-03-receipt-batch.

A receipt calculator prices two items and totals a grade batch. Currency uses decimal, quantities use int, discounts are bounded, and invalid external text is parsed safely.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A receipt calculator prices two items and totals a grade batch. Currency uses decimal, quantities use int, discounts are bounded, and invalid external text is parsed safely. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Add 59/60,69/70,79/80 and 89/90 grade boundary checks.
- Use a for loop to print array indexes; compare with foreach.
- Add an empty grade list guard so division by zero cannot occur.
- Demonstrate integer division $5/2$ versus decimal division $5m/2$.

Acceptance checks and expected output

- $19.90 \times 2 \times 0.90$ produces 35.82.
- Malformed quantities are rejected with TryParse.
- Grade 90 is A and 89 is B.
- Negative quantities fail; discount remains between 0 and 1.

```
Receipt total: 35.82
```

```
Mean grade: 80.00
```

```
Grade band: B
```

```
Parse quantity 'two': False
```

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Activity 04 — Methods and console MVC for balance enquiries

LO2 · K2 · A2; deck slides 133–137. Folder: activities/activity-04-methods-mvc.

A bank requires console MVC: Model owns a non-negative balance, View formats output, Controller coordinates. Public members use PascalCase and local/private variables camelCase.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A bank requires console MVC: Model owns a non-negative balance, View formats output, Controller coordinates. Public members use PascalCase and local/private variables camelCase. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.
2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Add nullable account lookup and a view error message without moving business rules into the View.
- Replace recursion with iteration and compare the same boundary cases.
- Draw a sequence diagram for Controller → Model → View.

Acceptance checks and expected output

- The Model rejects negative initial balance.
- View owns formatting; Controller holds no account rule.
- Factorial(0) is 1 and Factorial(5) is 120.
- Public method/property names follow PascalCase.

DEM0-001: balance 125.50
Factorial(5): 120

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Topic 3 — Classes, objects and design controls

Deck slides 138–176; maps to K3, A3 and LO3.

Access modifiers define a visibility boundary

Slides 139–140. protected internal allows same assembly OR derived type; private protected requires same assembly and containing/derived type.

```
public class Base {  
    protected internal int shared;  
    private protected int family;  
    private int secret = 10;  
    public int ReadSecret() => secret;  
}
```

Failure to diagnose: Visibility is not identity verification or authorization.

Design decision: Expose the smallest necessary surface; a top-level class defaults internal and a nested class defaults private.

Original source coverage: slides 68, 69, 70, 71, 72, 73. These subjects have been recomposed and corrected in the mechanism above.

Classes model identity and controlled state

Slides 141–142. Construction starts with a valid state and a private setter limits mutation paths.

```
sealed class Stock {  
    public int Count { get; private set; }  
    public Stock(int initial) {  
        if(initial<0) throw new ArgumentOutOfRangeException(nameof(initial));  
        Count=initial;  
    }  
}
```

Failure to diagnose: A public unchecked setter can admit negative stock.

Design decision: Put invariants in the component; let UI controls collect and display values.

Original source coverage: slides 96, 97, 98. These subjects have been recomposed and corrected in the mechanism above.

Constructors initialize complete objects

Slides 143–144. Constructor chaining reuses one validation path.

```
sealed class Product {  
    public string Name { get; }  
    public Product() : this("Unspecified") { }  
    public Product(string name) {  
        if(string.IsNullOrEmpty(name)) throw new ArgumentException("Name required");  
        Name=name;  
    }  
}
```

Failure to diagnose: Duplicated constructor logic may validate one path but miss another.

Design decision: Require every essential invariant at the creation boundary.

Original source coverage: slides 99, 100. These subjects have been recomposed and corrected in the mechanism above.

Finalizers are not reliable cleanup

Slides 145–146. using disposes a resource at the end of scope, including exceptional exits.

```
// Deterministic disposal:  
using var writer = new StreamWriter("stock.txt");  
writer.WriteLine("SKU-001,10");  
//
```

Type() is a nondeterministic finalizer.

Failure to diagnose: Waiting for a finalizer may leave a file handle open for an unknown time.

Design decision: Use IDisposable/using for resources; implement a finalizer only for justified unmanaged ownership.

Original source coverage: slides 101. These subjects have been recomposed and corrected in the mechanism above.

Static state has process-level lifetime

Slides 147–148. A stateless policy is shared without per-object state.

```
static class FinePolicy {  
    public const decimal DailyRate = 0.50m;
```

```

    public static decimal Calculate(int overdueDays)
    => Math.Max(0,overdueDays) * DailyRate;
}

```

Failure to diagnose: Mutable static counters persist across unrelated tests and callers.

Design decision: Prefer stateless static functions or explicit scoped service instances.

Original source coverage: slides 102, 103. These subjects have been recomposed and corrected in the mechanism above.

Inheritance and base construction

Slides 149–150. A derived constructor supplies the base state.

```

abstract class Person(string name) {
    public string Name { get; } = name;
}
sealed class Student(string name) : Person(name) { }

```

Failure to diagnose: Inheriting only to reuse a few lines may create a false substitutability relationship.

Design decision: Use inheritance for a real is-a contract; use composition for has-a dependencies.

Original source coverage: slides 104, 105. These subjects have been recomposed and corrected in the mechanism above.

One class base, multiple interfaces

Slides 151–152. A class may implement multiple contracts but inherit one class base.

```

interface IReadable { string Read(); }
interface IWritable { void Write(string value); }
class Store : IReadable, IWritable {
    public string Read() => "demo";
    public void Write(string value) { }
}

```

Failure to diagnose: Two base classes are not supported in a C# class declaration.

Design decision: Use interfaces to decouple callers from interchangeable components.

Original source coverage: slides 106, 124. These subjects have been recomposed and corrected in the mechanism above.

Overloads select by compile-time signature

Slides 153–154. Argument types choose different overloads before execution.

```

static string Show(int quantity) => $"count {quantity}";
static string Show(decimal price) => $"price {price:F2}";
Console.WriteLine(Show(10));
Console.WriteLine(Show(10m));

```

Failure to diagnose: Two methods cannot differ only by return type.

Design decision: Keep overload meanings consistent and avoid ambiguous optional parameters.

Original source coverage: slides 107, 108. These subjects have been recomposed and corrected in the mechanism above.

Virtual dispatch chooses runtime behaviour

Slides 155–156. A base-typed reference dispatches to the derived override.

```
abstract class Policy { public abstract string Band(decimal mean); }
sealed class Standard : Policy {
    public override string Band(decimal mean) => mean>=90?"A":"B";
}
```

// Policy p = new Standard(); p.Band(85) returns B.

Failure to diagnose: new hides a member; it does not provide the same override contract.

Design decision: Test components through the shared interface/base type.

Original source coverage: slides 109, 110, 111. These subjects have been recomposed and corrected in the mechanism above.

Operator overloads must preserve meaning

Slides 157–158. A user type can define meaningful addition while remaining immutable.

```
readonly record struct Money(decimal Amount) {
    public static Money operator +(Money a, Money b)
        => new(a.Amount+b.Amount);
}
```

// new Money(2m)+new Money(3m) -> Amount 5m

Failure to diagnose: Surprising operator semantics make callers misread the code.

Design decision: Prefer explicit named methods when the operation is not naturally an operator.

Original source coverage: slides 112, 113, 114. These subjects have been recomposed and corrected in the mechanism above.

Properties express controlled mutation

Slides 159–160. Validation computes the candidate before assigning, preserving the old state on rejection.

```
public int Stock { get; private set; }
public void AdjustStock(int delta) {
    int next = checked(Stock + delta);
    if(next<0) throw new InvalidOperationException("Insufficient stock");
    Stock=next;
}
```

Failure to diagnose: Subtracting first and validating afterwards leaves an invalid object.

Design decision: Make rejected changes atomic at the local component level.

Composition injects interchangeable services

Slides 161–162. Dependencies are visible in construction and can be replaced by local mocks.

```
// OrderProcessor(PaymentService payment,
//   NotificationService notification, string logPath)
// calls payment before notification.
```

Failure to diagnose: Creating a network client deep inside business logic makes deterministic failure testing difficult.

Design decision: Keep the classroom adapter synthetic; document the interface needed for a real service.

An interface is a caller-visible contract

Slides 163–164. A caller can use either student without knowing its concrete class.

```
interface IGradable {  
    decimal CalculateAverage();  
    string GetGradeBand();  
}
```

// Process both policies through IGradable.

Failure to diagnose: A shared method name without a common contract leaves the caller tied to concrete types.

Design decision: Test substitutability through the interface and state input/error guarantees.

Controls map to data and behaviour

Slides 165–166. Each control has a specific property, rule and output responsibility.

```
// Numeric input -> AddGrade(value)  
// Student selector -> Student.Name  
// Results grid -> average and band  
// Confirmation -> successful model change only
```

Failure to diagnose: A visually suitable control still needs range checks and error feedback.

Design decision: Choose controls by valid input range, allowed choices, readability and accessibility.

Activity 05 — Catalogue classes, constructors and encapsulated properties

LO3 · K3 · A3; deck slides 167–171. Folder: activities/activity-05-product-objects.

Administrators add product records. Product constructors enforce invariants and the catalogue owns its collection. Categories are typed; invalid records cannot enter the catalogue.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: Administrators add product records. Product constructors enforce invariants and the catalogue owns its collection. Categories are typed; invalid records cannot enter the catalogue. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Add stock adjustment checks for valid/invalid deltas.
- Implement lookup by exact name using ordinal comparison and return nullable when absent.
- Explain why a private setter alone is not authentication or authorization.

Acceptance checks and expected output

- An invalid name, price, stock or category is rejected.
- The collection is exposed read-only and stock changes through a method.
- Three named categories are available.
- Valid add increments the count and displays confirmation.

Added Keyboard; catalogue count: 1

Keyboard | Electronics | 49.90 | stock 10

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Activity 06 — Interfaces, inheritance and polymorphic grade controls

LO3 · K3 · A3; deck slides 172–176. Folder: activities/activity-06-inheritance-grades.

A school needs extensible grade classification. A base Student owns validated grades; GradedStudent and HonoursStudent implement a shared IGradable contract and demonstrate virtual dispatch.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A school needs extensible grade classification. A base Student owns validated grades; GradedStudent and HonoursStudent implement a shared IGradable contract and demonstrate virtual dispatch. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.
2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

`dotnet --list-sdks`
3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.
4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Add 89/90 and 84/85 boundary cases for the two policies.
- Draw an editable UML class diagram with one base class and the interface contract.
- List suitable GUI controls (numeric grade input, student selector, results grid) and map each to its property/method; the supplied console app is the portable implementation.

Acceptance checks and expected output

- All grades are within 0–100.
- Empty grade collection fails with a meaningful error.
- The same mean 85 is B for standard and A for honours.
- Both students are processed through IGradable.

```
Mean 85.00; band B
```

```
Mean 85.00; band A
```

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Topic 4 — Functionality and interoperability

Deck slides 177–213; maps to K4, A4 and LO4.

Struct construction and default values

Slides 178–179. Modern C# allows a public parameterless struct constructor; default still zero-initializes.

```
readonly struct Measurement {  
    public int Value { get; }  
    public Measurement() { Value=10; }  
}  
// new Measurement().Value -> 10  
// default(Measurement).Value -> 0
```

Failure to diagnose: Assuming default invokes the custom constructor gives an invalid invariant.

Design decision: Use structs for small value semantics and design for valid default state.

Original source coverage: slides 116, 117, 118, 119, 120. These subjects have been recomposed and corrected in the mechanism above.

Enums need validation at external boundaries

Slides 180–181. Casting a number can produce an unnamed enum value.

```
enum Category { Electronics, Clothing, Food }  
var category = (Category)99;  
Console.WriteLine(Enum.IsDefined(category)); // False
```

Failure to diagnose: An enum declaration alone does not reject invalid imported numeric values.

Design decision: Validate parsed enum values and document how names/numbers are exchanged.

Original source coverage: slides 121, 122. These subjects have been recomposed and corrected in the mechanism above.

Regex validates format, not domain semantics

Slides 182–183. Anchors require the entire SKU; timeout bounds the match work.

```
using System.Text.RegularExpressions;  
bool ok = Regex.IsMatch("SKU-001", @"^ASKU-[0-9]{3}$",  
    RegexOptions.CultureInvariant, TimeSpan.FromMilliseconds(100));
```

Failure to diagnose: An unanchored pattern accepts extra prefix/suffix text.

Design decision: Follow format checks with domain checks such as uniqueness and allowed stock.

Original source coverage: slides 132, 133, 134. These subjects have been recomposed and corrected in the mechanism above.

Catch specific exceptions at a meaningful boundary

Slides 184–185. The format failure is handled while unrelated failures remain visible.

```
try { int quantity = int.Parse("ten"); }  
catch (FormatException ex) {  
    Console.WriteLine($"Invalid quantity: {ex.GetType().Name}");  
}  
finally { /* release owned resources if needed */ }
```

Failure to diagnose: A broad empty catch suppresses defects and may claim success incorrectly.

Design decision: Use TryParse for normal bad input; use catch for exceptional failures.

Original source coverage: slides 135, 136, 137, 138, 139. These subjects have been recomposed and corrected in the mechanism above.

Custom exception and rethrow retain diagnostic context

Slides 186–187. A named domain exception distinguishes an invariant failure.

```
sealed class StockException(string message) : Exception(message);  
// Within a catch block:  
// throw; preserves the original stack.  
// throw ex; restarts the stack at this line.
```

Failure to diagnose: Replacing the original exception without context hides the failing call chain.

Design decision: Keep useful context and avoid sensitive data in exception messages.

Original source coverage: slides 140, 141. These subjects have been recomposed and corrected in the mechanism above.

Files and streams define an I/O contract

Slides 188–189. The writer encodes text over a byte stream and disposal releases handles.

```
using var stream = new FileStream("stock.txt", FileMode.Create, FileAccess.Write);  
using var writer = new StreamWriter(stream);  
writer.WriteLine("SKU-001,10");
```

Failure to diagnose: Open/create/append modes have different overwrite behaviour.

Design decision: Choose mode and encoding deliberately; never let untrusted input choose an arbitrary path.

Original source coverage: slides 142, 143, 144, 145. These subjects have been recomposed and corrected in the mechanism above.

State transitions gate notifications

Slides 190–191. The outcome of one component controls what the next component may do.

```
// Pending -> PaymentProcessing  
// success -> PaymentSuccessful -> Confirmed -> notify  
// reject/exception -> PaymentFailed -> no notify  
// cancellation -> Cancelled -> no payment/notify
```

Failure to diagnose: Notifying before checking payment produces a false confirmation.

Design decision: Use explicit states and test every failure/cancellation path.

Interoperability needs a data exchange contract

Slides 192–193. Names, types and allowed values are agreed by producer and consumer.

```
sealed record ProductDto(string Sku,string Name,int Quantity);  
// JSON: {"Sku":"SKU-001","Name":"Keyboard","Quantity":10}  
// Quantity is a number, not "ten".
```

Failure to diagnose: Successful transport with a mismatched schema is still an integration failure.

Design decision: Test round-trip equality, missing values, invalid types and version changes.

JSON success does not prove a valid domain object

Slides 194–195. Deserialization and business validation are distinct acceptance stages.

```
var p = JsonSerializer.Deserialize<ProductDto>(json);
// Then validate p != null, SKU format, nonblank name,
// and Quantity >= 0 before adding to catalogue.
```

Failure to diagnose: A syntactically valid JSON object can contain negative stock.

Design decision: Reject invalid data before mutating the live collection.

Logs link failures across components

Slides 196–197. The correlation ID connects the payment failure to its resulting order state.

```
// ORDER-004: PaymentProcessing
// ORDER-004: payment exception
// Result: PaymentFailed; notified False
```

Failure to diagnose: Logging only error text without an ID makes concurrent records hard to connect.

Design decision: Record identity, stage and outcome without exposing credentials or personal data.

Functionality indicators have an observable definition

Slides 198–199. The numerator/denominator and exercised paths define what the figure proves.

```
// Success rate = passed cases / attempted cases.
// 4 correct paths / 4 exercised paths = 100%.
// This is test-suite evidence, not a production SLA.
```

Failure to diagnose: A single happy-path pass cannot establish all-path correctness.

Design decision: Report expected/actual outputs, error rate and timing under a stated workload.

Resource lifetime is part of interoperability

Slides 200–201. Disposing the writer flushes/closes the file before the next component reads it.

```
using(var writer = new StreamWriter(path))
    writer.Write(JsonSerializer.Serialize(item));
var reloaded = Import(File.ReadAllText(path));
```

Failure to diagnose: Reading before closing/flushing can observe incomplete output.

Design decision: Own and dispose each resource at a clear boundary; verify file sharing/mode assumptions.

Repeated calls require a separate idempotency design

Slides 202–203. A single-call test does not prove safe retries.

```
// First Process(ORDER-001) -> notification.
// Second Process(ORDER-001) currently repeats it.
// Proposed guard: completed order-ID record.
```

Failure to diagnose: A transient retry can duplicate an external effect.

Design decision: Document the sample limitation and test deduplication before claiming production readiness.

Activity 07 — Order states, payment failure and notification guards

LO4 · K4 · A4; deck slides 204–208. Folder: activities/activity-07-order-state-exceptions.

A food-delivery processor must never confirm a failed payment or notify a cancelled order. Mock services exercise success, rejection and thrown exceptions; status transitions are written to a local log.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A food-delivery processor must never confirm a failed payment or notify a cancelled order. Mock services exercise success, rejection and thrown exceptions; status transitions are written to a local log. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Read the local event log and build a state transition table.
- Add an invalid-state test directly against NotificationService.Send.
- State a limitation: repeat calls are not idempotent. Propose an order-ID deduplication design and its tests without claiming it exists.

Acceptance checks and expected output

- Success confirms and notifies exactly once in a single call.
- Rejected or thrown payments fail without notification.
- Cancelled orders bypass payment and notification.
- Log contains the order ID and failure path.

```
ORDER-001: Confirmed; notified True
ORDER-002: PaymentFailed; notified False
Log: output/order-log.txt
```

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Activity 08 — JSON contracts, enum parsing, regex and file lifetime

LO4 · K4 · A4; deck slides 209–213. Folder: activities/activity-08-json-interoperability.

A warehouse imports synthetic product data using a documented JSON contract. Reject mismatched types and negative stock, preserve UTF-8 names and prove export/import round-trip interoperability.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A warehouse imports synthetic product data using a documented JSON contract. Reject mismatched types and negative stock, preserve UTF-8 names and prove export/import round-trip interoperability. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.
2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Create a contract table with field, type, required value, valid example and failure.
- Add missing-field and JSON-null tests and document serializer defaults.
- Add a schema-version field and explain backward compatibility.
- Explain why using `StreamWriter` deterministically and a finalizer is unsuitable.

Acceptance checks and expected output

- A valid record survives JSON export/import unchanged.
- Quantity is a JSON integer, not text; stock cannot be negative.
- SKU uses exactly SKU- plus three digits.
- UTF-8 names and zero stock are preserved.

```
Imported SKU-001; stock 10
Round-trip equal: True
Export: output/product.json
```

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Topic 5 — Design documentation and handover

Deck slides 214–242; maps to K5, A5 and LO5.

Namespaces and using resolve symbols

Slides 215–216. Namespaces organize names; using makes names available without a qualified prefix.

```
using System.Text.Json;
namespace Warehouse.Design;
public sealed record ProductDto(string Sku,int Quantity);
```

Failure to diagnose: Two imported types with the same name create ambiguity.

Design decision: Use qualified names or aliases; a namespace is not a deployment/security boundary.

Original source coverage: slides 125, 126, 127. These subjects have been recomposed and corrected in the mechanism above.

Conditional compilation changes the built artifact

Slides 217–218. Only the selected branch is compiled into the assembly.

```
#define DEMO
#if DEMO
Console.WriteLine("Synthetic local data");
#else
Console.WriteLine("Configured production adapter");
#endif
```

Failure to diagnose: Testing only a debug branch leaves another compiled configuration unverified.

Design decision: Keep configuration explicit; #define does not create a normal runtime variable.

Original source coverage: slides 128, 129, 130, 131. These subjects have been recomposed and corrected in the mechanism above.

XML comments describe the public API contract

Slides 219–220. Summary, parameter and return tags connect the API to the documented rule.

```
/// <summary>Charges 0.50 per overdue calendar day.</summary>
/// <param name="due">Due date.</param>
/// <param name="returned">Actual return date.</param>
/// <returns>Non-negative fine in SGD.</returns>
public static decimal Calculate(DateOnly due, DateOnly returned)
    => Math.Max(0, returned.DayNumber - due.DayNumber) * 0.50m;
```

Failure to diagnose: A comment that contradicts the implementation misleads the next developer.

Design decision: Keep comments, tests and specification IDs synchronized.

Documentation is generated from a compiler setting

Slides 221–222. The XML output is a real build artifact that can be inspected.

```
<GenerateDocumentationFile>true</GenerateDocumentationFile>
// Build emits Activity.xml beside the assembly.
// CS1591 identifies undocumented public API when enabled.
```

Failure to diagnose: A checked-in comment alone does not prove documentation generation is configured.

Design decision: Inspect the emitted XML and document any deliberate warning policy.

Requirement traceability reaches executable evidence

Slides 223–224. A reviewer can follow one user requirement into a method and a concrete test.

```
// R4 fine policy
// -> FineCalculator.Calculate
// -> due 10 Sep, return 13 Sep
// -> expected 1.50, actual 1.50
```

Failure to diagnose: A mapping that ends at a class name cannot prove the required behaviour.

Design decision: Record requirement, method, test input, expected/actual output and evidence location.

Dependency notes identify real package boundaries

Slides 225–226. The handover distinguishes shipped code dependencies from authoring tools.

```
// Target: .NET 10 (net10.0).
// BCL: System.IO, System.Text.Json, Regex.
// Third-party NuGet packages: none.
// Optional editor/AI tools are not runtime dependencies.
```

Failure to diagnose: Invented package versions imply software the project does not contain.

Design decision: Read csproj/project references and report exact real dependencies.

A design decision record documents the tradeoff

Slides 227–228. The chosen control, alternative and consequences explain why the implementation has its shape.

```
// Decision: encapsulate stock behind Add/Remove.  
// Alternative: unrestricted public setter.  
// Reason: preserve nonnegative stock on every path.  
// Consequence: callers handle rejected operations.
```

Failure to diagnose: Documenting only the chosen class omits the reason and migration limits.

Design decision: Tie decisions to requirements and revisit them when requirements change.

A class library moves code across a project boundary

Slides 229–230. The build resolves a reusable local component as a separate referenced assembly.

```
<ProjectReference Include="../Stock.Core/Stock.Core.csproj" />  
// App calls public StockService from Stock.Core.  
// Internal types remain within their assembly.
```

Failure to diagnose: Copying the same class into multiple apps creates drifting implementations.

Design decision: Document the public API and rerun consumer tests after library changes.

Handover states practical limits

Slides 231–232. The receiver can see what the working sample proves and what must be engineered next.

```
// Sample: in-memory, single process, no user auth.  
// Data reset when application exits.  
// Future work: persistence, authorization, concurrency.  
// Each new feature requires a new contract/test.
```

Failure to diagnose: A passing classroom demo is insufficient evidence for production safety under concurrency.

Design decision: Include run instructions, known limits, dependencies, test evidence and requirement mapping.

Activity 09 — Library methods, XML documentation and specification traceability

LO5 · K5 · A5; deck slides 233–237. Folder: activities/activity-09-library-xml-docs.

A council library requires book search, member registration, loan processing and a fine of 0.50 per overdue calendar day. Produce XML comments, dependency notes and an explicit requirement-to-method-to-test map.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A council library requires book search, member registration, loan processing and a fine of 0.50 per overdue calendar day. Produce XML comments, dependency notes and an explicit requirement-to-method-to-test map. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Build XML documentation and locate bin/Debug/net10.0/Activity.xml.
- Add duplicate loan and duplicate member tests.
- Complete a dependencies table with SDK target, built-in namespaces and no third-party package assumption.
- Add summary/param/returns/exception tags to a new return-book method.

Acceptance checks and expected output

- Book search is case-insensitive.
- Only registered members may loan an available book.
- Three overdue calendar days produce 1.50; early return produces zero.
- Generated report maps R1–R4 to named public methods.

Search matches: 1

Fine: 1.50

Report: output/design-report.txt

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Activity 10 — Documented stock component and handover evidence

LO5 · K5 · A5; deck slides 238–242. Folder: activities/activity-10-design-handover.

A warehouse requires a reusable stock service that prevents negative stock, reports changes and can be traced to R-STOCK-01. Deliver a design decision, dependency inventory, public API comments and executable evidence.

Files and prerequisites

Activity.csproj, Program.cs, README.md/PDF, requirements.md/PDF, prompts.md/PDF, evidence.md/PDF and expected-output.txt. .NET 10 SDK; no API keys or third-party packages.

Vibe-coding prompts

Design prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A warehouse requires a reusable stock service that prevents negative stock, reports changes and can be traced to R-STOCK-01. Deliver a design decision, dependency inventory, public API comments

and executable evidence. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

Review prompt

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

Evidence prompt

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Detailed step-by-step procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.

2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.

4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.

8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.

9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
```

```
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Move StockService into a documented class library and add a project reference; rerun the same tests.
- Draw a class and sequence diagram and map R-STOCK-01 to the public API.

- Create a decision record comparing in-memory state and persistence, with tradeoffs and a future migration test.
- Archive your actual test outputs and one reviewed change in the handover.

Acceptance checks and expected output

- $10 - 3 + 2$ produces stock 9.
- Overselling is rejected without changing existing stock.
- Negative and overflow mutations are rejected.
- Handover states the contract, requirement trace, dependencies and limitations.

Stock after remove/add: 9
Handover: output/handover.txt

Troubleshooting

SDK target failure: install/select .NET 10. Compiler error: fix the first file/line diagnostic. Assertion failure: reproduce the input and review the invariant. I/O error: use a writable local output folder and dispose handles. An AI C++/package mismatch: reject the unsupported edit and request a C#-only diff.

Evidence and handover

Complete evidence.md: actual SDK/build output; normal/invalid/boundary cases; requirement → source method → test; one failure/repair; AI review decision or manual review; limitations and next change.

Assessment preparation and submission

WA has five open-ended knowledge questions mapped one-to-one to K1–K5; PP has five practical tasks mapped one-to-one to A1–A5. Each paper is 60 minutes and is completed individually using approved open-book resources. Attend the assessment session, complete the provided documents and submit them to <https://lms-tms.tertiaryinfotech.com/>; obtain assessor feedback and sign the assessment summary record. Candidate papers are distributed through the LMS; answer keys are trainer-only. This course does not prepare for an external certification exam, so no external practice-exam slide is included.

Source register and C++ comparison boundary

Source	Role / location
Original PPT v6.0	reference/WSQ - Master Trainer Slides - Programming Methodologies in C# - v6.pptx
Approved proposal / TSC mapping	reference/extracted-original/CP_TIPL_Csharp_v1.1.txt
Registration page	https://www.tertiarycourses.com.sg/wsqa-vibe-coding-with-c-sharp.html
Microsoft first C# learning path	https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-1/
C# interactive supplementary exercises	https://www.learncs.org/
Microsoft C# tour and reference	https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/
Microsoft C# learning portal	https://dotnet.microsoft.com/en-us/learn/csharp
Microsoft beginner videos (legacy UI)	https://learn.microsoft.com/en-us/shows/csharp-fundamentals-for-absolute-beginners/
C++ comparison only	https://www.w3schools.com/cpp/default.asp
C++ comparison only	https://www.geeksforgeeks.org/cpp/c-plus-plus/

Microsoft documentation supplies current C#/.NET contracts. learncs.org is supplementary interactive practice; legacy Microsoft videos are conceptual references with old UI. The two C++ links support language comparison only. C++ syntax, standard libraries, native ownership and build commands are not substituted for C# project code. Technical examples here are original course-specific worked artifacts, not copied third-party tutorials.

Content source: original PPT v6.0 and CP/TSC mapping. Design source: current Tertiary house visual components. Concept hero: built-in Imagegen, courseware/assets/cover-hero-v7.png, 13 September 2026; illustration of a reviewed developer workbench, no generated labels.

Glossary

Term	Meaning
Invariant	Rule that must hold before and after a permitted operation.
Contract	Allowed input, output, state and error behaviour between caller and component.
Assembly	Compiled .NET deployment/type boundary.
Interoperability	Components exchange data and coordinate outcomes using agreed contracts.
Nullable	A type annotation/value form that represents possible absence.
Polymorphism	A common contract dispatches behaviour to the actual implementation.
Traceability	Requirement ID → source component/method → executable test/evidence.